

Міністерство освіти і науки України

Прикарпатський національний університет імені В.Стефаника

Факультет математики та інформатики
Кафедра інформаційних технологій
Інформатика і програмування

Лабораторна робота №1

Тема роботи: Налаштування середовища розробки та запуск програм з
використанням MPI

Варіант №23

Виконав: Жолобчук В.В.
Група ІІЗ-31
Дата: 19 лютого 2026р.
Викладач: Лазарович І.М.

Івано-Франківськ
2026

Завдання (Варіант №8):

1. Налаштувати середовище розробки (WSL Ubuntu + Visual Studio Code).
2. Виконати запуск тестової програми **hello.cpp** із кількістю процесів $N = (23\%10) = 3$.
3. Виконати запуск програми **icpi.cpp** для обчислення числа π з кількістю інтервалів 2.1×10^9 на 2, 4 та 8 процесах.

Скріншоти виконання програми hello.cpp та обчислення числа π

```
vasil@HP-G8-250:~/mpi_lab1$ mpiexec -n 3 ./hello
Hello world
Hello world
Hello world
vasil@HP-G8-250:~/mpi_lab1$ mpirun -np 2 ./icpi
Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) 2100000000
pi is approximately 3.14159265358979274433, Error is 0.0000000000000037166
wall clock time = 12.593569
Enter the number of intervals: (0 quits) 0

[proxy:0@HP-G8-250] HYDU_sock_write (lib/utils/sock.c:250): write error (Broken pipe)
[proxy:0@HP-G8-250] HYD_pmcd_pmip_control_cmd_cb (proxy/pmip_cb.c:525): unable to write to downstream stdin
[proxy:0@HP-G8-250] HYDT_dmxu_poll_wait_for_event (lib/tools/demux/demux_poll.c:76): callback returned error status
[proxy:0@HP-G8-250] main (proxy/pmip.c:122): demux engine error waiting for event
[mpiexec@HP-G8-250] control_cb (mpiexec/pmiserv_cb.c:280): assert (!closed) failed
[mpiexec@HP-G8-250] HYDT_dmxu_poll_wait_for_event (lib/tools/demux/demux_poll.c:76): callback returned error status
[mpiexec@HP-G8-250] HYD_pmci_wait_for_completion (mpiexec/pmiserv_pmci.c:173): error waiting for event
[mpiexec@HP-G8-250] main (mpiexec/mpiexec.c:260): process manager error waiting for completion
vasil@HP-G8-250:~/mpi_lab1$ mpirun -np 4 ./icpi
Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) 2100000000
pi is approximately 3.14159265358979274433, Error is 0.0000000000000037166
wall clock time = 12.571816
Enter the number of intervals: (0 quits) 0

[proxy:0@HP-G8-250] HYDU_sock_write (lib/utils/sock.c:250): write error (Broken pipe)
[proxy:0@HP-G8-250] HYD_pmcd_pmip_control_cmd_cb (proxy/pmip_cb.c:525): unable to write to downstream stdin
[proxy:0@HP-G8-250] HYDT_dmxu_poll_wait_for_event (lib/tools/demux/demux_poll.c:76): callback returned error status
[proxy:0@HP-G8-250] main (proxy/pmip.c:122): demux engine error waiting for event
[mpiexec@HP-G8-250] control_cb (mpiexec/pmiserv_cb.c:280): assert (!closed) failed
[mpiexec@HP-G8-250] HYDT_dmxu_poll_wait_for_event (lib/tools/demux/demux_poll.c:76): callback returned error status
[mpiexec@HP-G8-250] HYD_pmci_wait_for_completion (mpiexec/pmiserv_pmci.c:173): error waiting for event
[mpiexec@HP-G8-250] main (mpiexec/mpiexec.c:260): process manager error waiting for completion
vasil@HP-G8-250:~/mpi_lab1$ mpirun -np 8 ./icpi
Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) Enter the number of intervals: (0 quits) 2100000000
pi is approximately 3.14159265358979274433, Error is 0.0000000000000037166
wall clock time = 15.801438
Enter the number of intervals: (0 quits) █
```

Результати показали що програма працює найшвидше на 4 процесах (12.57 с),
При запуску на 8 процесах час виконання зріс до 15.80 с.

Лістинг програми ісрі.crr

```
#include "mpi.h"

#include <stdio.h>
#include <math.h>

double f(double a)
{
    return (4.0 / (1.0 + a*a));
}

int main(int argc, char *argv[])
{
    int done = 0, myid, numprocs;
    long long int n, i;
    double PI25DT = 3.141592653589793238462643;
    long double mypi, pi, h, sum, x;
    double startwtime = 0.0, endwtime;
    int namelen;
    char processor_name[MPI_MAX_PROCESSOR_NAME];

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    MPI_Get_processor_name(processor_name, &namelen);

    while (!done) {
        if (myid == 0) {
            fprintf(stdout, "Enter the number of intervals: (0 quits) ");
            fflush(stdout);
            if (scanf("%lld", &n) != 1) {
                fprintf(stdout, "No number entered; quitting\n");
                n = 0;
            }
            startwtime = MPI_Wtime();
        }
        MPI_Bcast(&n, 1, MPI_LONG_LONG_INT, 0, MPI_COMM_WORLD);
        if (n == 0)
            done = 1;
        else {
            h = 1.0L / n;
            sum = 0.0;
            for (i = myid + 1; i <= n; i += numprocs) {
                x = h * ((double)i - 0.5);
                sum += f(x);
            }
            mypi = h * sum;
            MPI_Reduce(&mypi, &pi, 1, MPI_LONG_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

            if (myid == 0) {
```

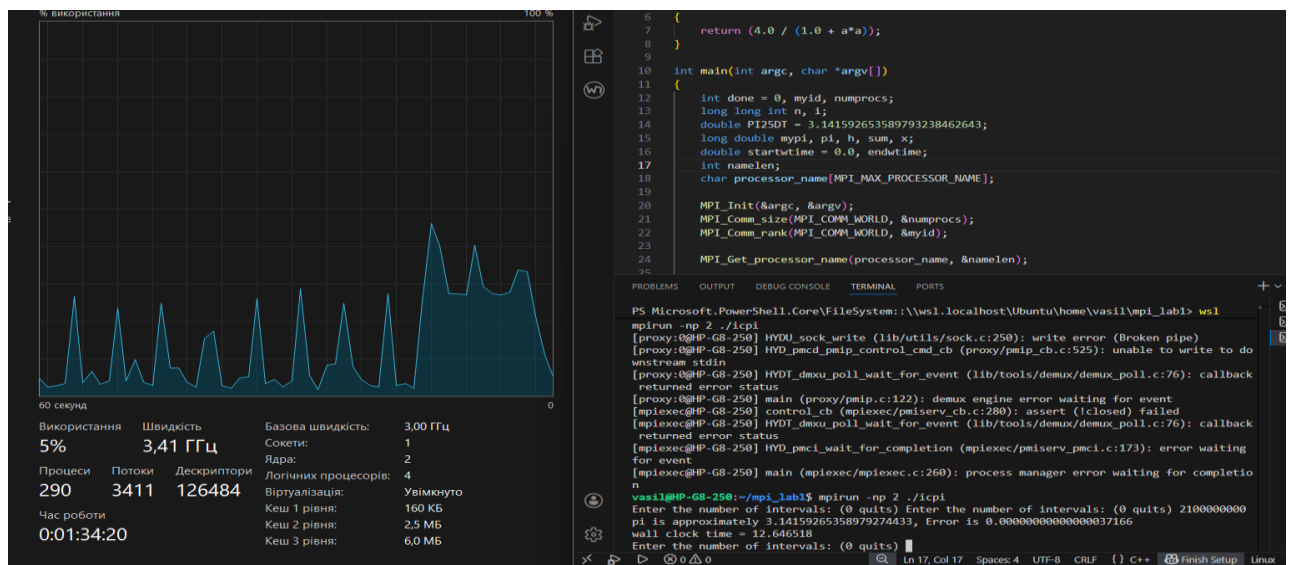
```

        printf("pi is approximately %.20Lf, Error is %.20Lf\n", pi, fabs(pi
- PI25DT));

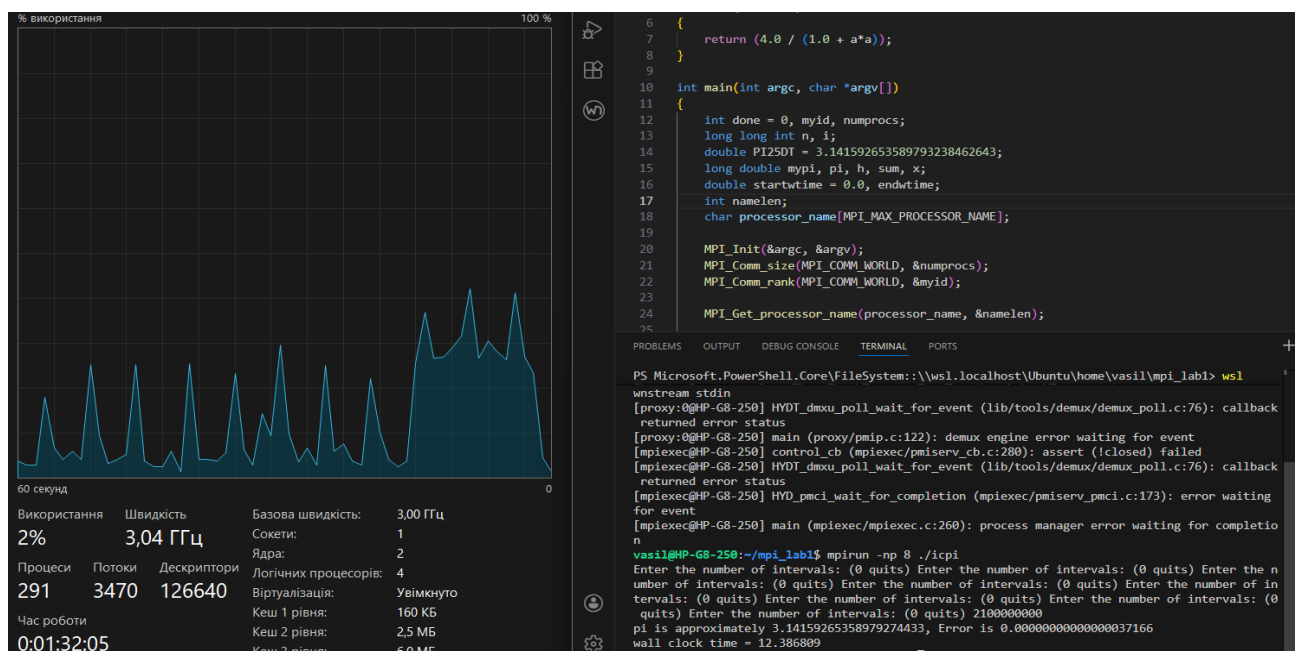
        endwtime = MPI_Wtime();
        printf("wall clock time = %f\n", endwtime - startwtime);
        fflush(stdout);
    }
}
MPI_Finalize();
return 0;
}

```

Скріншот виконання програми ісрі.crr, в яких суміщено вікно завантаження процесора/ядер під час обчислення з 2 процесами



Скріншот виконання програми ісрі.crr, в яких суміщено вікно завантаження процесора/ядер під час обчислення з 8 процесами



Системні ресурси

Для виконання лабораторної роботи використовувалася ноутбук з наступними характеристиками:

Процесор: 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz.

Кількість фізичних ядер: 2.

Кількість віртуальних ядер (логічних процесорів): 4.

Оперативна пам'ять: 8 ГБ

Висновки

У ході лабораторної роботи я повністю опанував процес налаштування середовища WSL та бібліотеки MPICH для розробки паралельних програм. На практиці вдалося перевірити, як процеси взаємодіють між собою через функції MPI_Bcast та MPI_Reduce.